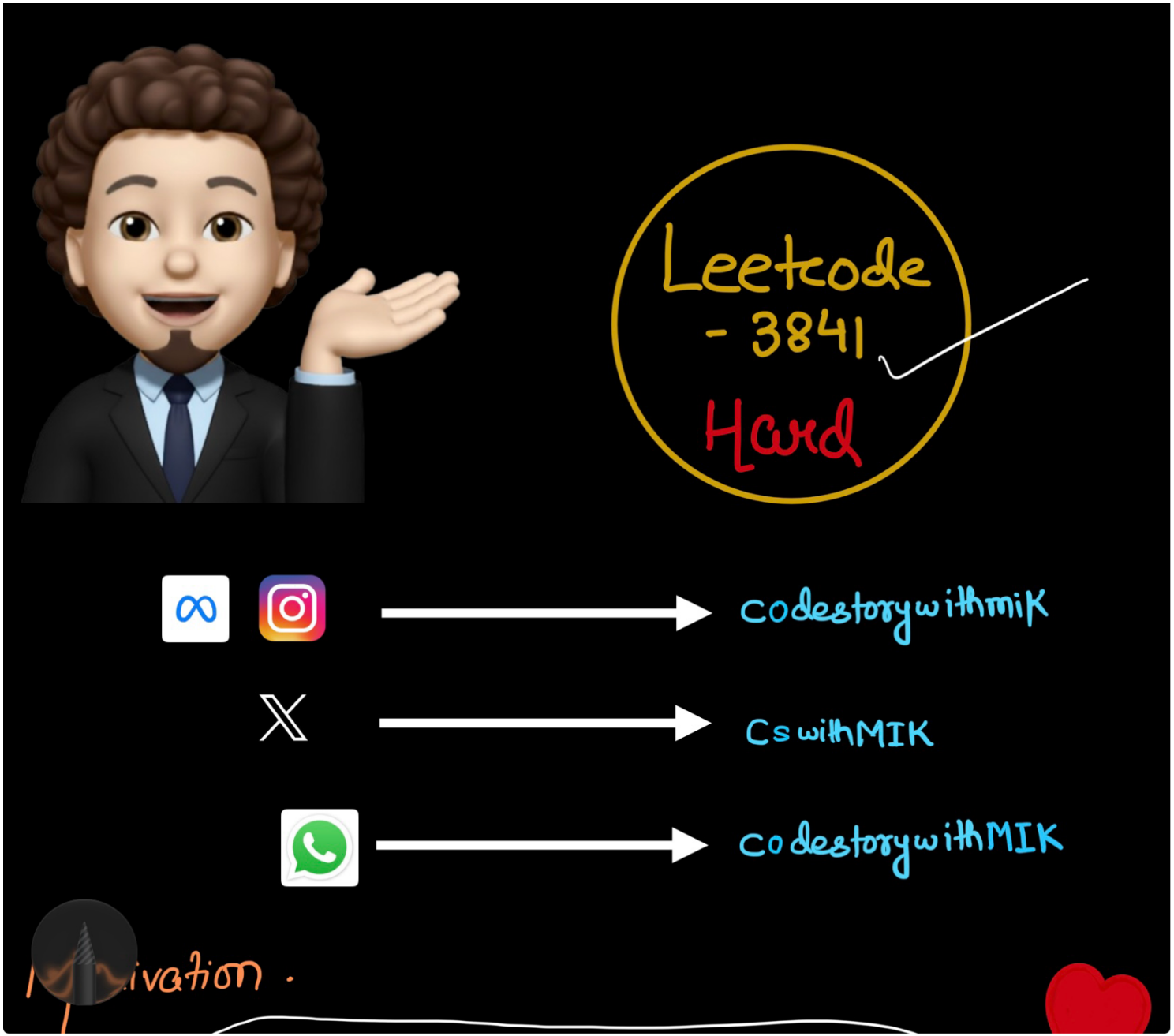
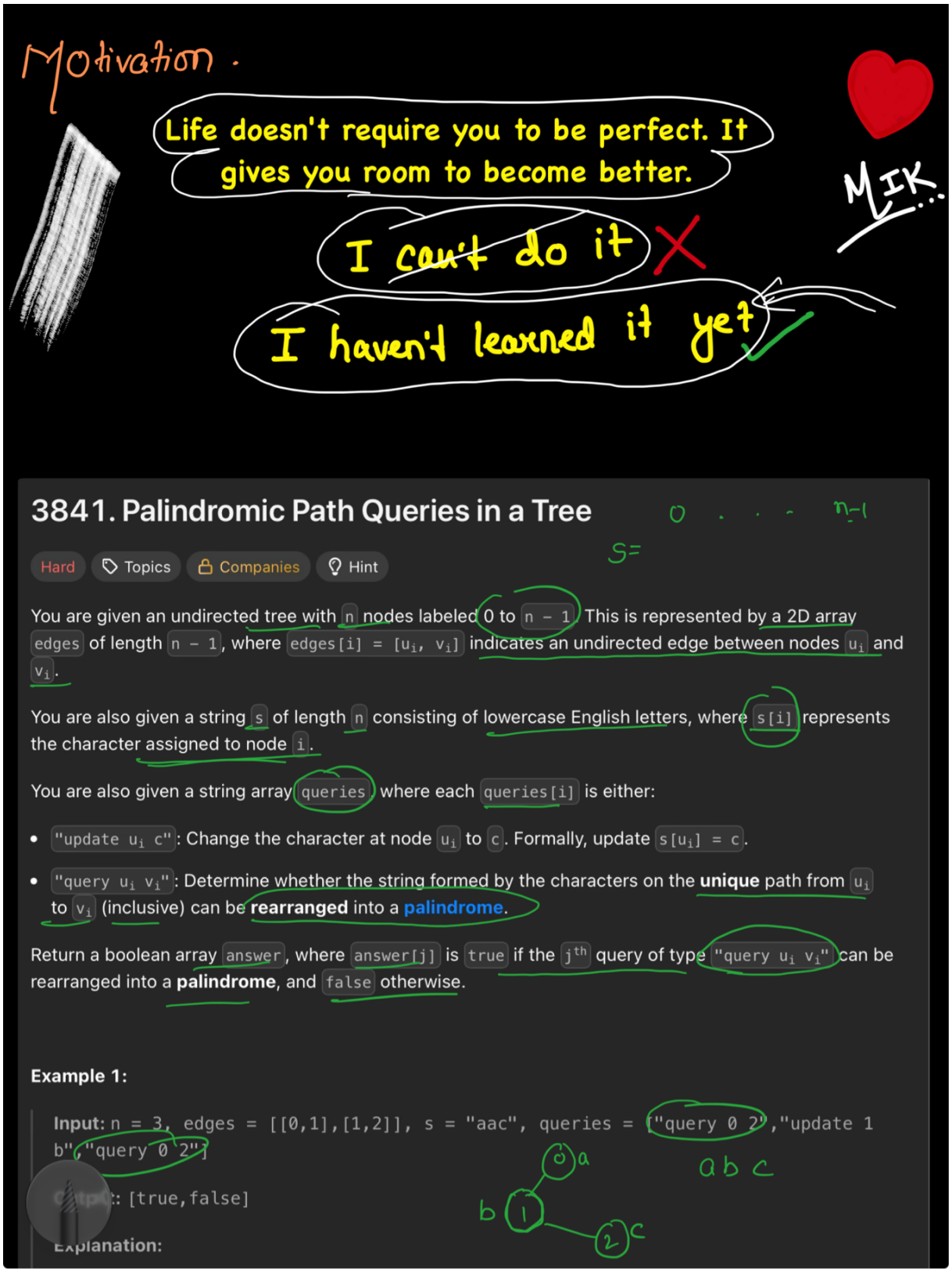

Leetcode
- 3841
Hard
codestorywithmiK
CswithMIK
codestorywithMIK



3841. Palindromic Path Queries in a Tree

Hard | Topics | Companies | Hint

You are given an undirected tree with `n` nodes labeled 0 to `n - 1`. This is represented by a 2D array `edges` of length `n - 1`, where `edges[i] = [u_i, v_i]` indicates an undirected edge between nodes `u_i` and `v_i`.

You are also given a string `s` of length `n` consisting of lowercase English letters, where `s[i]` represents the character assigned to node `i`.

You are also given a string array `queries` where each `queries[i]` is either:

- `"update u_i c"`: Change the character at node `u_i` to `c`. Formally, update `s[u_i] = c`.
- `"query u_i v_i"`: Determine whether the string formed by the characters on the **unique** path from `u_i` to `v_i` (inclusive) can be **rearranged** into a palindrome.

Return a boolean array `answer`, where `answer[j]` is `true` if the `j^th` query of type `"query u_i v_i"` can be rearranged into a **palindrome**, and `false` otherwise.

Example 1:

Input: `n = 3, edges = [[0,1],[1,2]], s = "aac", queries = ["query 0 2","update 1 b","query 0 2"]`

Output: `[true,false]`

Explanation:

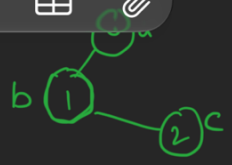
8:45 AM Wed 28 Oct

Output: `[true,false]`

Explanation:

- `"query 0 2"`: Path `0 → 1 → 2` gives `"aac"`, which can be rearranged to form `"aca"`, a palindrome. Thus, `answer[0] = true`.
- `"update 1 b"`: Update node 1 to `'b'`, now `s = "abc"`.
- `"query 0 2"`: Path characters are `"abc"`, which cannot be rearranged to form a palindrome. Thus, `answer[1] = false`.

Thus, `answer = [true, false]`.





Constraints:

- `1 <= n == s.length <= 5 * 10^4`
- `edges.length == n - 1`
- `edges[i] = [u_i, v_i]`
- `0 <= u_i, v_i <= n - 1`
- `s` consists of lowercase English letters.
- The input is generated such that `edges` represents a valid tree.
- `1 <= queries.length <= 5 * 10^4`
  - `queries[i] = "update u_i c"` or
  - `queries[i] = "query u_i v_i"`
  - `0 <= u_i, v_i <= n - 1`
  - `c` is a lowercase English letter.

$5 \cdot 10^4 * 5 \cdot 10^4$

$25 \cdot 10^8$

$2.5 \cdot 10^9$

"query 0 2" For

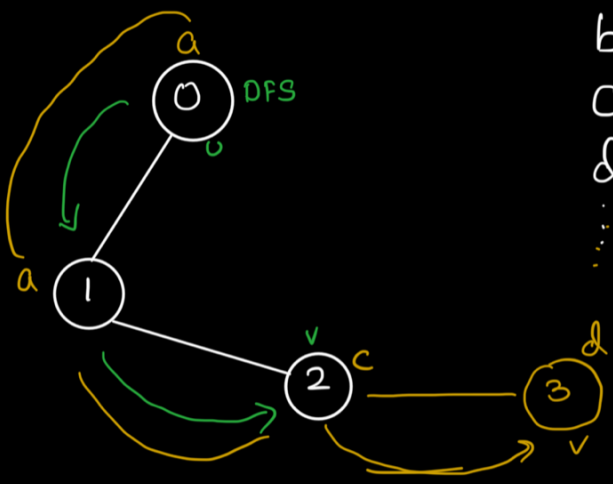
O(n)

O(q * n) T.L.E.

b → 1
c → 1

Improving our Approach

a = 0001
b = 0010
c = 0100
d = 1000



a → 0001
a → 0001

"a a c"

a a b c

a b b a

a c b a

Palindrome → At most 1 character has odd frequency.

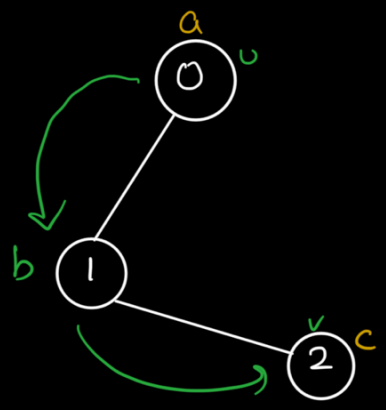
Brute Force

n = 3

edges = [(0,1), (1,2)]

s = "a a c"

b



O(n)

"query u 0 v 2" → True

Queries ⇒ "update 1 b" → O(1)

"query u 0 v 2" → False

O(n)

"abc"

a → 1

b → 1

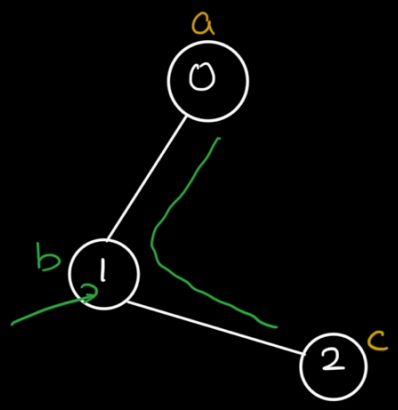
c → 1

Thought Process

n = 3

edges = [(0,1), (1,2)]

s = "a a c"



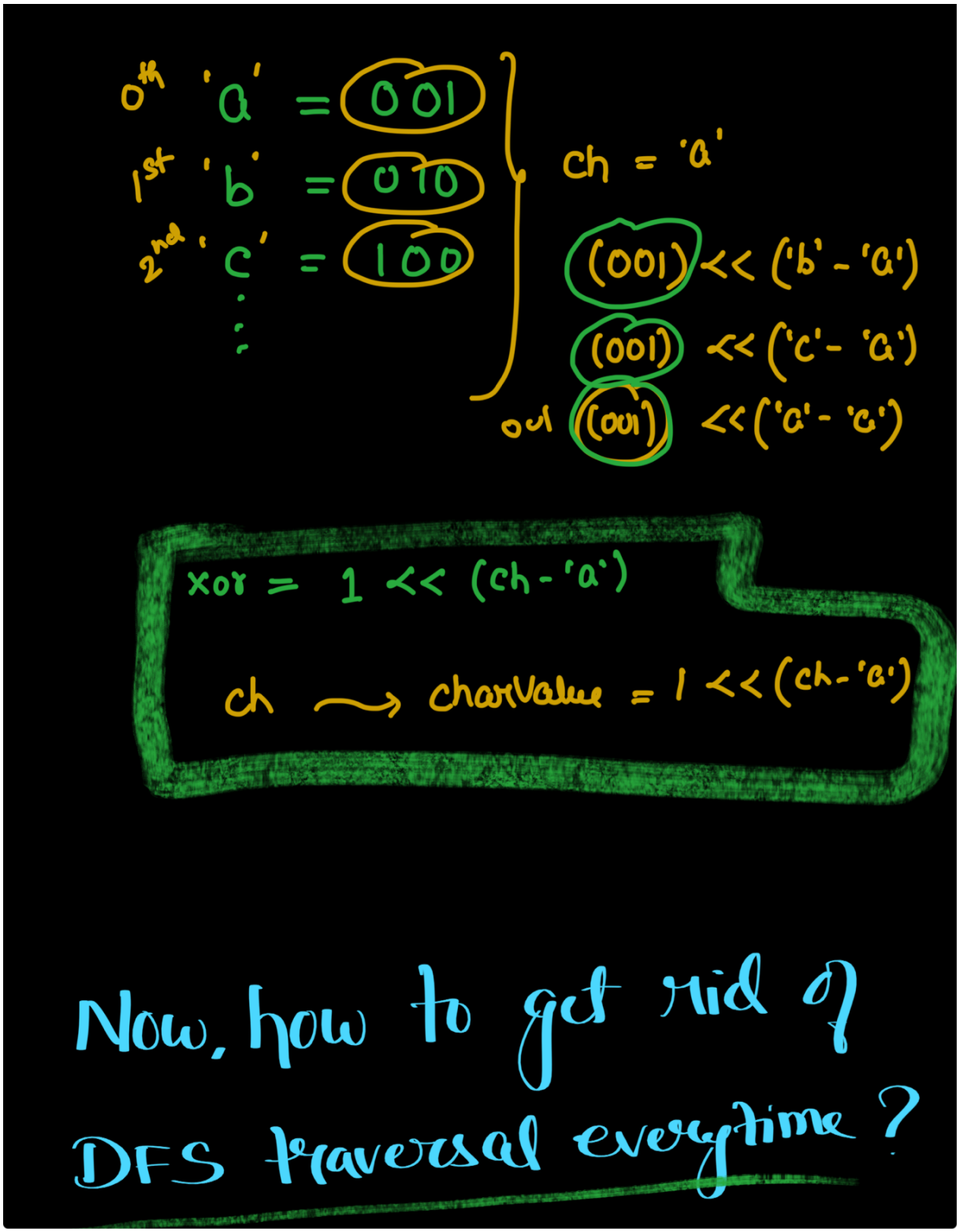
Queries ⇒

- "query 0 2" → T
- "update (1) (b)"
- "query 0 2" → F

How will you check a path can be a palindrome or not ?

"a a c"

a b b a


0th 'a' = 001
1st 'b' = 010
2nd 'c' = 100
ch = 'a'
(001) << ('b' - 'a')
(001) << ('c' - 'a')
out (001) << ('a' - 'a')
xor = 1 << (ch - 'a')
ch ⟶ charValue = 1 << (ch - 'a')
Now, how to get rid of
DFS traversal everytime ?

a → 0001
a → 0001
c → 0100

aca

x = 0100

& (x-1) = 0011

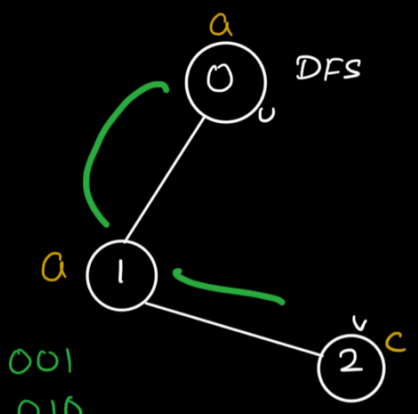
0000

---

n = 3

edges = [(0,1), (1,2)]

s = "a a c"



a = 001
b = 010
c = 100

Queries ⇒ "query u v" "query 0 2"

"update 1 b"

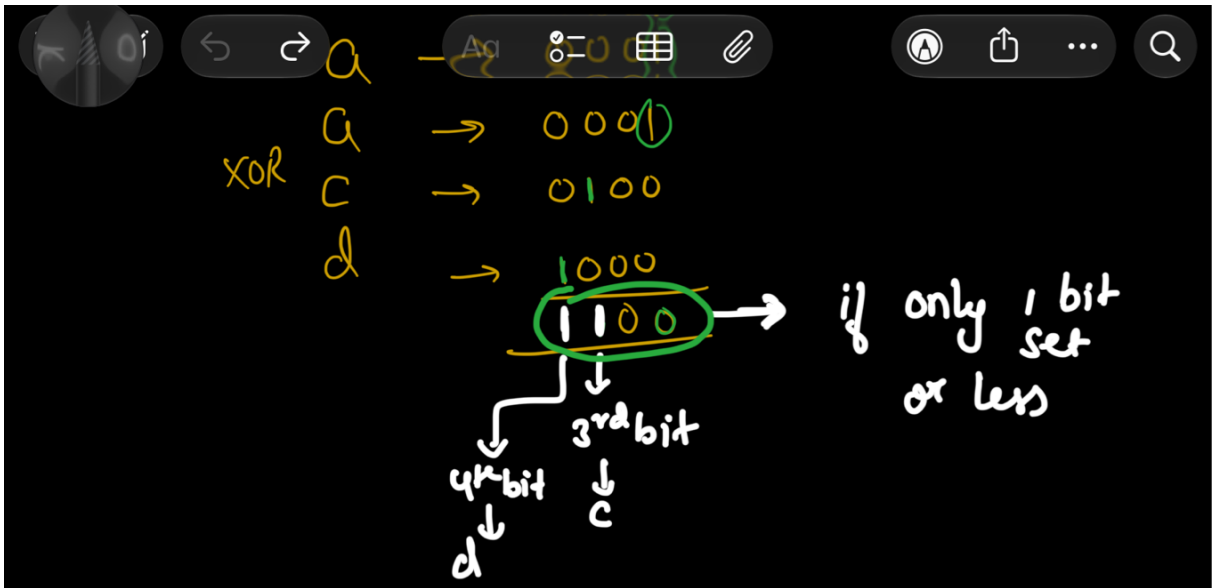
"query 0 2"

x = (001) ∧ (001) ∧ (100)

x = 100

if (x & (x-1) == 0)

True.



on the fly, calculate XOR of charValue

= Final XOR value = x

if $(x \,\&\, (x-1) == 0)$

can be a palindrome.

$x = 01\,00$

$(x-1) = 0011$

$1000 \neq 0$

① Path Sum (root to a Node) } → LCA
Path Xor (root to a Node) }

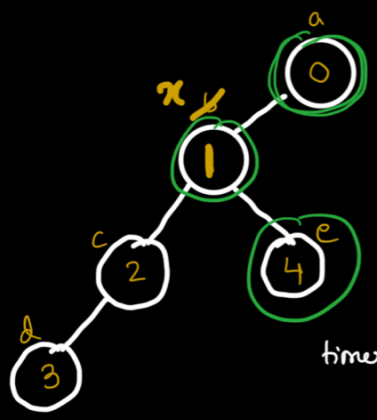
② O(1) Ancestor (U) → V ??

③ Sum/max/min/count → Euler Tour

⋮

DFS

update = 1, x



a∧b∧c∧d

timer = 7 8 9

node =	0	1	2	3	4
tin =	0	1	2	3	6

node =	0	1	2	3	4
tout =	9	8	5	4	7

time =	0	1	2	3	4	5	6	7	8	9
EulerTour =	a	b x	c	d	d	c	e	e	b x	a

result = -

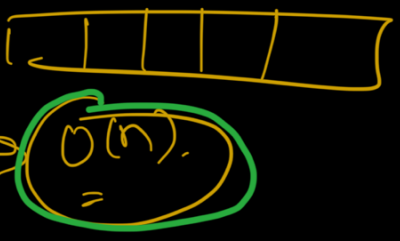
if (result & (result-1) == 0)

True;

update ① , 'x' ⟶ slow

we have to keep updating

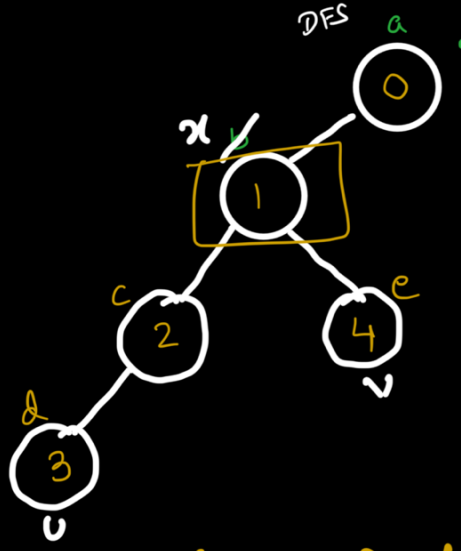
ROOT TO NODXOR



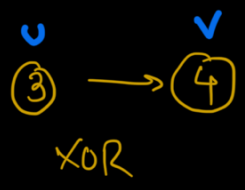
O(n).

Euler Tour

① Path Sum (root to a Node) } → LCA



$(a \wedge b \wedge c \wedge d)$
$\wedge$
$(a \wedge b \wedge e)$



$(root \to u) = (a \wedge b \wedge c \wedge d) \wedge$

$(root - v) = (a \wedge b \wedge e)$

$= c \wedge d \wedge e \wedge b$

$(u, v) = (root\ To\ u) \wedge (root\ To\ v) \wedge LCA(u, v)$

root To Node Xor =

0	1	2	3	4
a	$a \wedge b$	$a \wedge b \wedge c$	$a \wedge b \wedge c \wedge d$	$a \wedge b \wedge e$
a	b	c	d	e

Query (3,4) = $(a \wedge b \wedge c \wedge d)$ $\wedge$ $(a \wedge b \wedge e)$ $\wedge$ $LCA(3,4)$

$(a \wedge b \wedge c \wedge d)$: $O(1)$, u

$(a \wedge b \wedge e)$: $O(1)$, v

$LCA(3,4)$: $\log$, b

Result =

tout[node]

update (tin[node], char) ; } Seg
update (tout[node], char) ; } log



1. DFS
2. LCA → Binary Lifting
3. Segment Tree.
   - Build
   - qu
   - updat

① SegmentTree → ① Range Sum / XOR. → log

② update → log

index = 1

char = 'x'

① Query(U,V)

log → (0 . . . tin[U]) ^

log → (0 . . . tin[V]) ^ = mask

log → LCA(U,V) (mask & (mask-1)) == 0 Tip

② update (node, char)

S[node] = char ;

tin[node]
tout[node]

rootToNodeXor[3] → enter time of ③

tin[3] = 3

a ∧ b ∧ c ∧ d

EulerTour [0 .... tin[3]]

rootToNodeXor[4] = EulerTour [0 . . . tin[4]]

=

Query 3, 4 = RootToNode[3] ∧ RootToNode[4]

∧ LCA (3,4)

= EulerTour [0 ... tin[3]] ∧ EulerTour[0 ... tin[4]]

∧ LCA (3,4)